Dept. of Electronics and Electrical Communication Engineering

Indian Institute of Technology, Kharagpur

Digital Communications Laboratory

[EC39001]



Experiment – 6

Quadrature Phase Shift Keying (QPSK) Modulation & Demodulation (MATLAB Simulation)

Name : Aakarsh A

Roll No. : 22EC30001

Group No. : 6

Date of submission: 06-04-2025

Objectives:

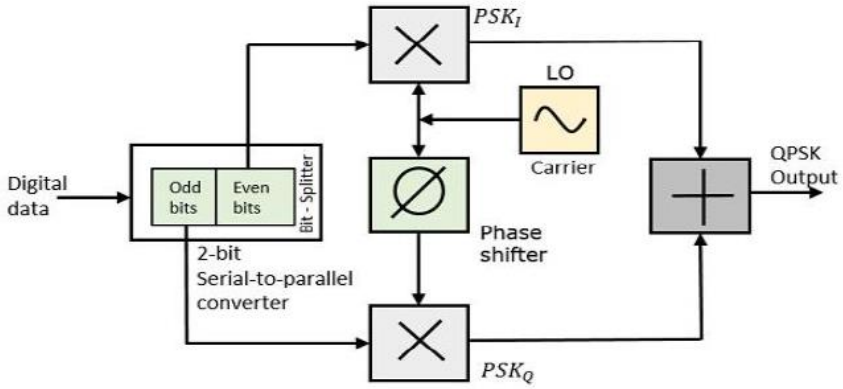
- To implement QPSK modulation and optimal demodulation for a random bit sequence in MATLAB/Python.

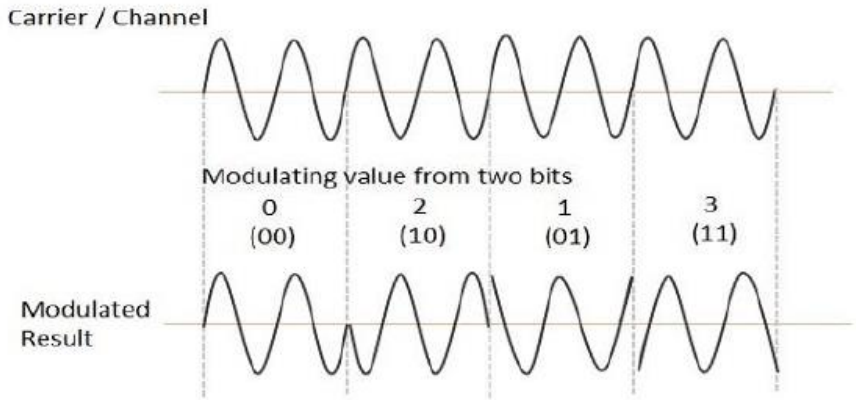
THEORY:

Phase Shift Keying: It is a digital modulation technique in which the phase of the carrier signal is changed by choosing either the sine or cosine signals at a particular time. PSK technique is widely used for wireless LANs and various other applications.

Quadrature Phase Shift Keying (QPSK): It is a variation of BPSK, and it is also a Double Side Band Suppressed Carrier (DSBSC) modulation scheme, which sends two bits of digital information at a time, which can be called bigits. Instead of converting digital bits into a series of digital streams, it converts them into bit pairs. This method decreases the data bit rate to half, allowing space for the other users.

QPSK Modulator: The QPSK modulator uses an even-odd bit splitter, two multipliers with a local oscillator, a 2-bit serial to parallel converter, and an adder circuit.





Expected output – Can have different constellations based on circuit configuration (depends on carrier and clock frequency).

Designed Logic & Code

QPSK Modulation

Below is the code, where the bit duration is taken as 0.1s, the sampling frequency at 1 kHz, and the samples_per_bit will be 100 samples per bit (for better accuracy and visualization). In this case, the total number of bits to be transmitted is 4096.

Binary 0 is mapped to -1 and 1 to +1. Then multiply the even NRZ and odd NRZ with sine and cosine carriers of half the frequency of data respectively.

```
%% Parameters
fs = 1000;               % Sampling frequency (Hz)
bit_duration = 0.1;      % Duration per bit (s)
samples_per_bit = fs * bit_duration;
num_bits = 4096;         % Number of bits (must be even for QPSK)
T_total = num_bits * bit_duration;

%% Generate random binary data
binary_sequence = randi([0, 1], 1, num_bits);

%% Create time-aligned binary sequence for plotting
plot_bits = 20;
binary_sequence_plot = repelem(binary_sequence(1:plot_bits), samples_per_bit);
t_plot = (0:length(binary_sequence_plot)-1)/fs;

%% QPSK Modulation
even_bits = binary_sequence(1:2:end);
odd_bits = binary_sequence(2:2:end);

% Convert to NRZ (±1)
nrz_even = 2 * even_bits - 1;
nrz_odd = 2 * odd_bits - 1;

f_carrier = 5;
t_symbol = (0 : 1/fs : 2*bit_duration - 1/fs);

% Modulate
qpsk_signal = zeros(1, num_bits/2 * length(t_symbol));
I_component = zeros(1, num_bits/2 * length(t_symbol));
Q_component = zeros(1, num_bits/2 * length(t_symbol));

for i = 1:length(nrz_even)
    idx = (i-1)*length(t_symbol) + (1:length(t_symbol));
    I_component(idx) = nrz_even(i) * cos(2*pi*f_carrier*t_symbol);
    Q_component(idx) = nrz_odd(i) * sin(2*pi*f_carrier*t_symbol);
    qpsk_signal(idx) = I_component(idx) + Q_component(idx);
end
```

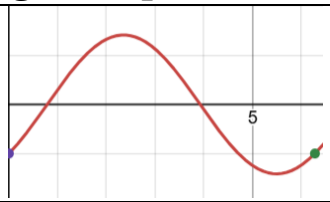
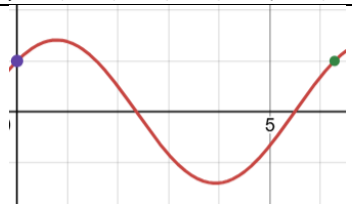
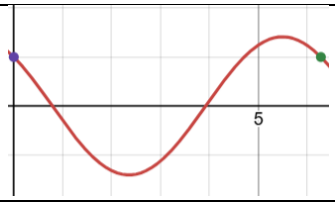
```
%% Plot original and modulated signals
figure(1);
subplot(4,1,1);
stairs(t_plot, binary_sequence_plot, 'k', 'LineWidth', 1.5);
title(['First ' num2str(plot_bits) ' Bits of Original Binary Sequence']);
xlabel('Time (s)'); ylabel('Amplitude'); ylim([-0.2 1.2]); grid on;

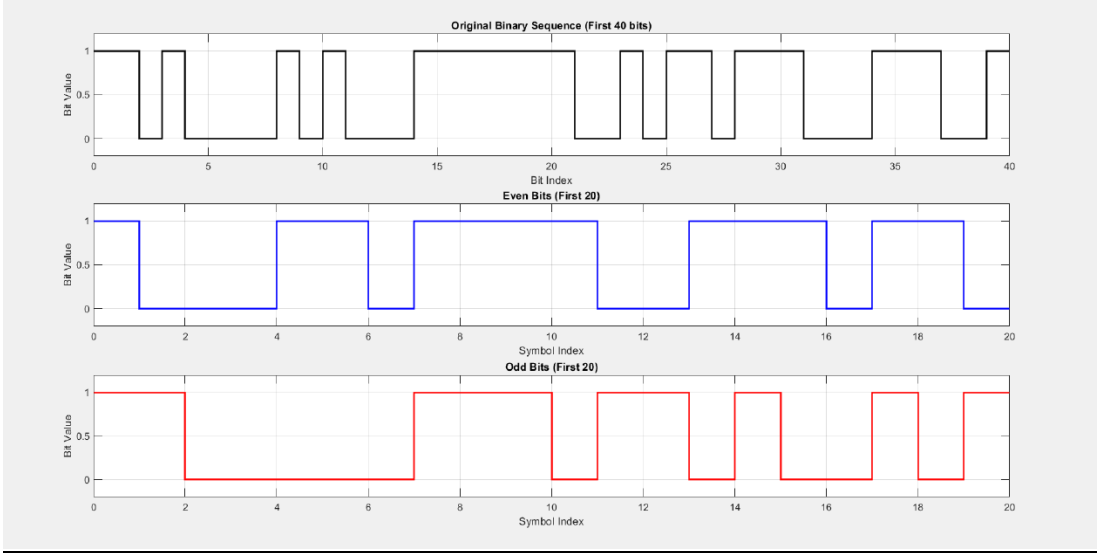
subplot(4,1,2);
plot((0:length(I_component(1:plot_bits*samples_per_bit/2))-1)/fs, ...
     I_component(1:plot_bits*samples_per_bit/2), 'b', 'LineWidth', 1.5);
title(['I Component (First ' num2str(plot_bits/2) ' Symbols)']);
xlabel('Time (s)'); ylabel('Amplitude'); grid on;

subplot(4,1,3);
plot((0:length(Q_component(1:plot_bits*samples_per_bit/2))-1)/fs, ...
     Q_component(1:plot_bits*samples_per_bit/2), 'r', 'LineWidth', 1.5);
title(['Q Component (First ' num2str(plot_bits/2) ' Symbols)']);
xlabel('Time (s)'); ylabel('Amplitude'); grid on;

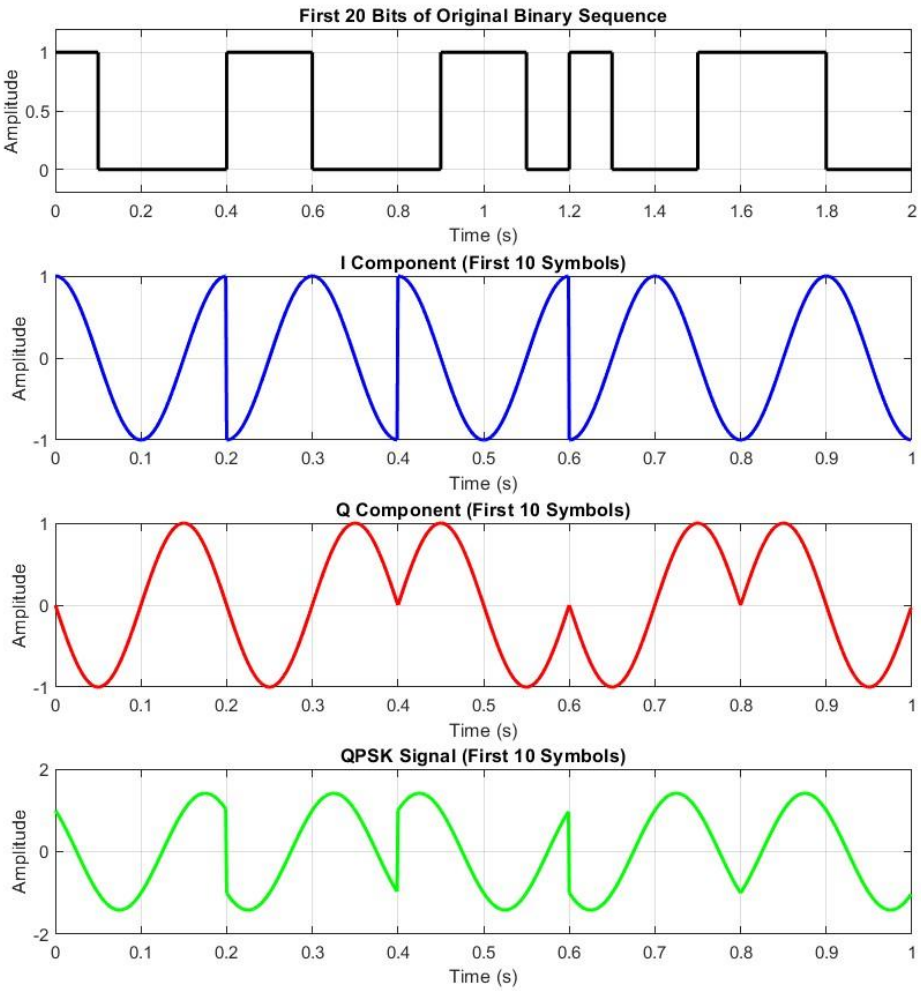
subplot(4,1,4);
plot((0:length(qpsk_signal(1:plot_bits*samples_per_bit/2))-1)/fs, ...
     qpsk_signal(1:plot_bits*samples_per_bit/2), 'g', 'LineWidth', 1.5);
title(['QPSK Signal (First ' num2str(plot_bits/2) ' Symbols)']);
xlabel('Time (s)'); ylabel('Amplitude'); grid on;
```

Constellations: For Clock frequency – 10 Hz and carrier frequency – 5 Hz

Bigit	QPSK Constellation	Signal Representation
10	Sin(x-45)	
00	Sin(x-135)	
11	Sin(x+45)	
01	Sin(x+135)	



Even bit and odd bit sequence for 20 bits.



QPSK Output with In-phase and Quadrature parts

Channel Effect & QPSK Demodulation:

For the demodulation, we get a modulated output with added noise from the channel. The noisy QPSK can be demodulated optimally using the minimum distance decoding principle.

To fully recover the message from the obtained signal, the carrier is generated at the destination, which is in phase with the modulated signal. It is implemented on Hardware using PLL for coherent detection. The signal is then multiplied with in-phase sine and cosine, and then the average is taken for each carrier interval. The averaged value optimally decides the value of the bit and effectively decodes the message.

The receiver can determine the pair of bits sent from the source by comparing the demodulated value to the nearest constellation point (called Euclidean distance criteria).

```
%% AWGN Channel & Constellation Plot (example)
snr_values = -80:2:20;
ser = zeros(size(snr_values));
ber = zeros(size(snr_values));  % <-- Added BER array
example_snr = -6;

% QPSK Constellation & Mapping
qpsk_constellation = [1+1j, 1-1j, -1+1j, -1-1j];
bit_pairs = [1 1; 1 0; 0 1; 0 0];

% Add noise for example constellation
noisy_signal = awgn(qpsk_signal, example_snr);

% Constellation Plot
num_symbols_plot = 100;
rx_symbols = zeros(1, num_symbols_plot);

for i = 1:num_symbols_plot
    idx = (i-1)*length(t_symbol) + (1:length(t_symbol));
    current_symbol = noisy_signal(idx);

    cos_carrier = cos(2*pi*f_carrier*t_symbol);
    sin_carrier = sin(2*pi*f_carrier*t_symbol);

    I_demod = sum(current_symbol .* cos_carrier) / length(t_symbol);
    Q_demod = sum(current_symbol .* sin_carrier) / length(t_symbol);

    rx_symbols(i) = I_demod + 1j*Q_demod;
end
```

```
137
138  figure(2);
139  subplot(2,1,1);
140  plot(real(qpsk_constellation), imag(qpsk_constellation), 'ro', 'MarkerSize', 10, 'LineWidth', 2);
141  grid on; axis([-1.5 1.5 -1.5 1.5]);
142  title('Ideal QPSK Constellation');
143  xlabel('In-Phase'); ylabel('Quadrature');
144
145  subplot(2,1,2);
146  plot(real(rx_symbols), imag(rx_symbols), 'b.');
147  hold on;
148  plot(real(qpsk_constellation), imag(qpsk_constellation), 'ro', 'MarkerSize', 10, 'LineWidth', 2);
149  grid on; axis([-1.5 1.5 -1.5 1.5]);
150  title(['Received Constellation (SNR = ' num2str(example_snr) ' dB)']);
151  xlabel('In-Phase'); ylabel('Quadrature');
152  legend('Received Symbols', 'Ideal Points', 'Location', 'Best');
```

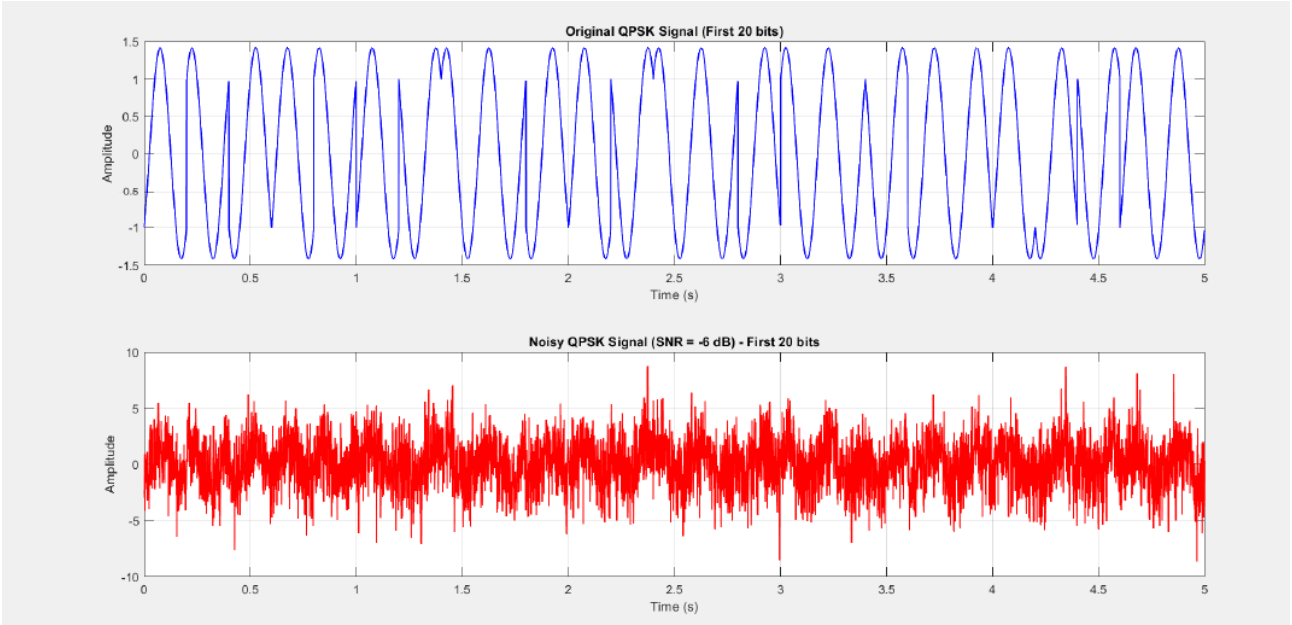
```
154  %% SER and BER vs SNR Calculation
155  num_symbols = num_bits / 2;
156  parfor snr_idx = 1:length(snr_values)
157      noisy_signal = awgn(qpsk_signal, snr_values(snr_idx), 'measured');
158      rx_symbols_temp = zeros(1, num_symbols);
159
160      for i = 1:num_symbols
161          idx = (i-1)*length(t_symbol) + (1:length(t_symbol));
162          current_symbol = noisy_signal(idx);
163
164          cos_carrier = cos(2*pi*f_carrier*t_symbol);
165          sin_carrier = sin(2*pi*f_carrier*t_symbol);
166
167          I_demod = sum(current_symbol .* cos_carrier) / length(t_symbol);
168          Q_demod = sum(current_symbol .* sin_carrier) / length(t_symbol);
169
170          rx_symbols_temp(i) = I_demod + 1j*Q_demod;
171      end
172
173      % Map original bits to transmitted symbols
174      tx_symbols = nrz_even + 1j * nrz_odd;
175
176      % Make decisions using minimum distance
177      decisions = zeros(1, num_symbols);
178      for i = 1:num_symbols
179          [~, idx_min] = min(abs(rx_symbols_temp(i) - qpsk_constellation));
180          decisions(i) = idx_min;
181      end
182
183      tx_indices = zeros(1, num_symbols);
184      for i = 1:num_symbols
185          tx_indices(i) = find(ismember(qpsk_constellation, tx_symbols(i)));
186      end
```

```
        % SER calculation
        ser(snr_idx) = sum(decisions ~= tx_indices) / num_symbols;

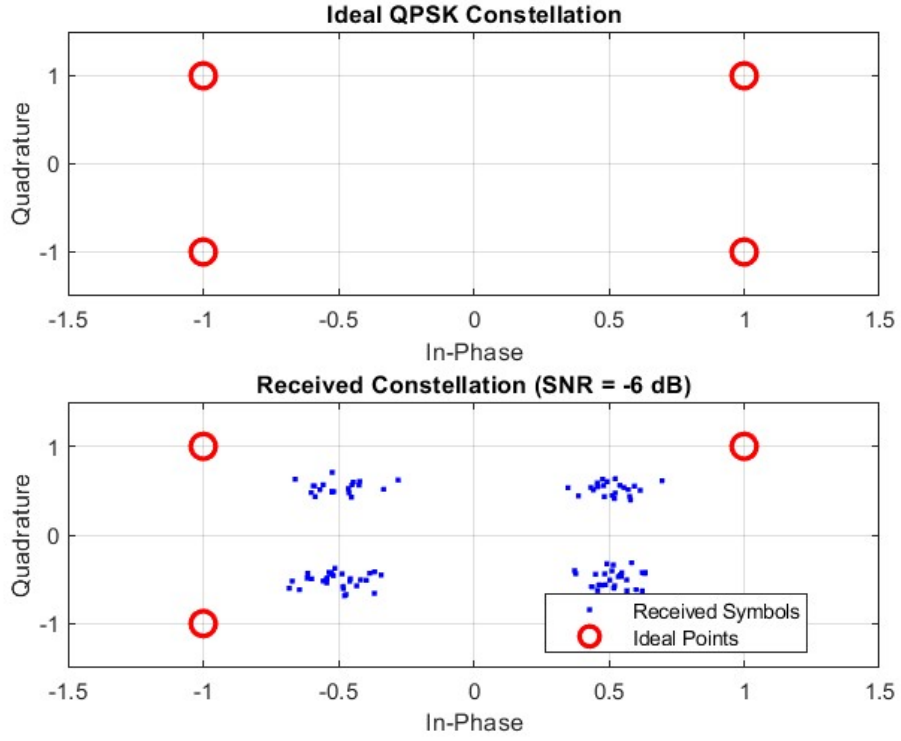
        % BER calculation
        received_bits = zeros(1, num_bits);
        for i = 1:num_symbols
            symbol_idx = decisions(i);
            received_bits(2*i-1:2*i) = bit_pairs(symbol_idx, :);
        end
        bit_errors = sum(binary_sequence ~= received_bits);
        ber(snr_idx) = bit_errors / num_bits;
    end

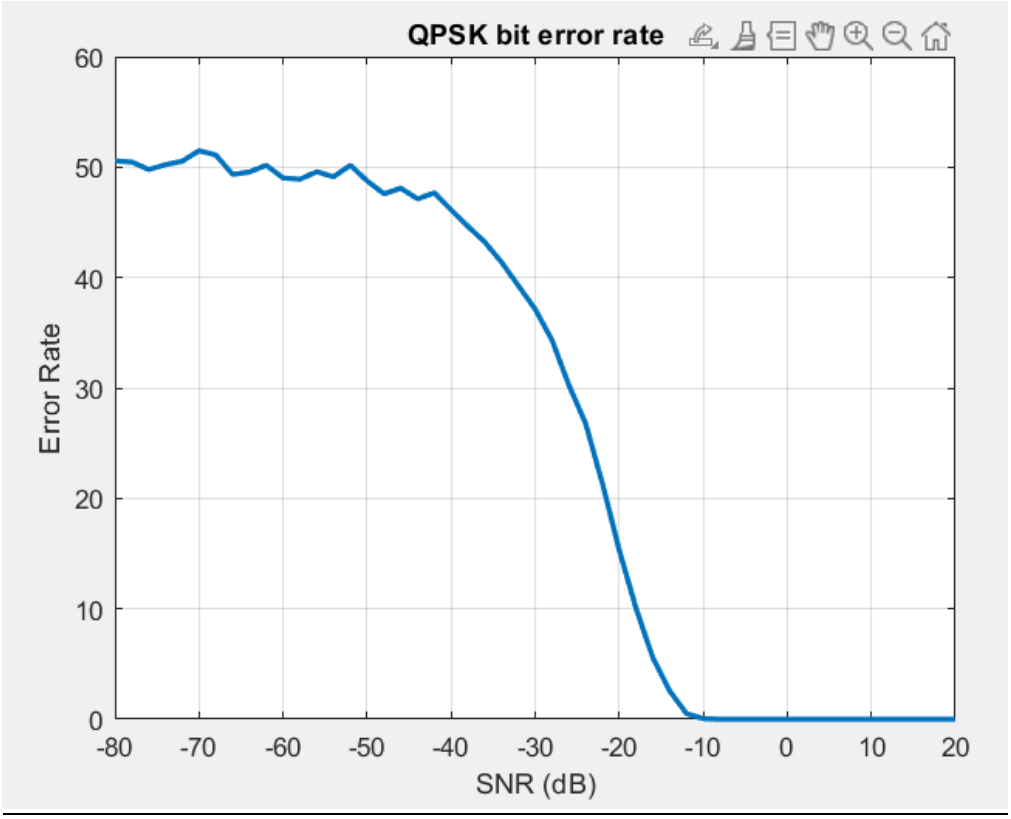
    %% Plot SER and BER
    figure(3) ;
    plot(snr_values, 100*ber, 'LineWidth', 2);
    grid on;
    xlabel('SNR (dB)');
    ylabel('Error Rate');
    title('QPSK bit error rate');

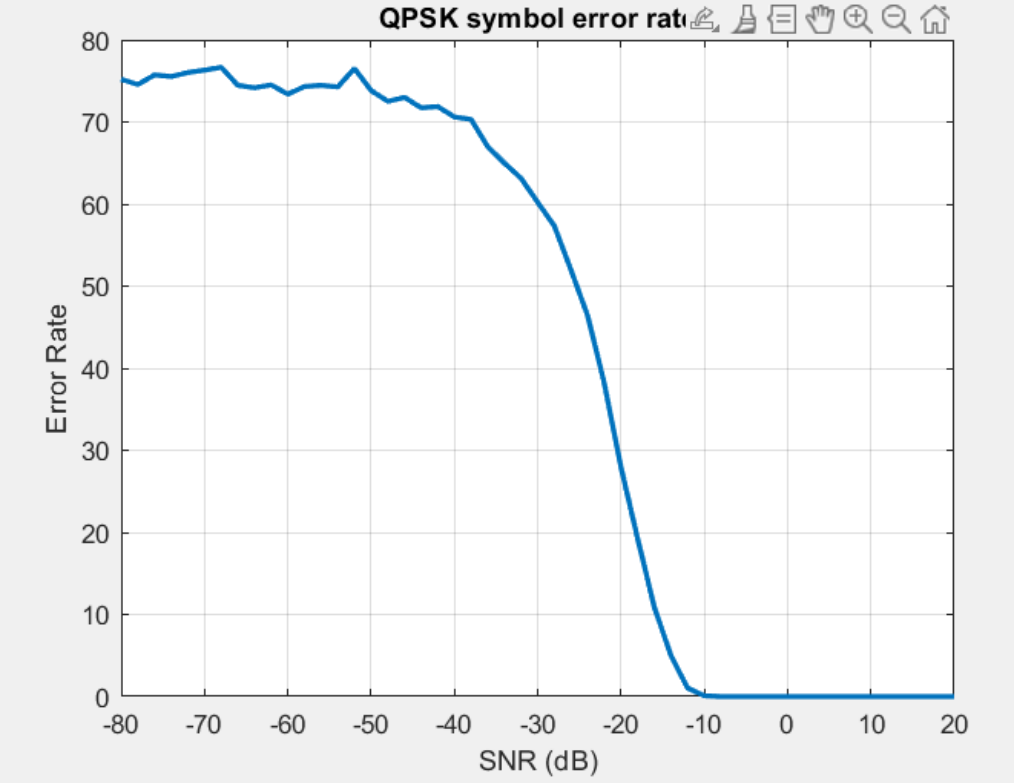
    figure(4) ;
    plot(snr_values, 100*ser, 'LineWidth', 2);
    grid on;
    xlabel('SNR (dB)');
    ylabel('Error Rate');
    title('QPSK symbol error rate');
```



Noisy QPSK


Ideal QPSK Constellation
Quadrature
In-Phase
Received Constellation (SNR = -6 dB)
Quadrature
In-Phase
Received Symbols
Ideal Points


QPSK bit error rate
Error Rate
SNR (dB)



Discussion:

- In this experiment, we have performed simulations of QPSK modulation and demodulation along with channel noise, giving us the idea and the principle behind both modulation and demodulation techniques.
- The binary sequence was generated using the 'randi' function within the limit [0 to 1] having a step size of 1, after which it was later separated into odd and even bits and converted to NRZ form
- Then it was multiplied with the sine and cosine terms and added to get the modulated output.
- AWGN was added to the modulated output to simulate real-time channel noise and its effect on the signal.
- We mapped each symbol representing two bits is mapped to a specific constellation point in the complex plane. The constellation consists of four points located at equal angular spacing (typically at 45◦, 135◦, 225◦, and 315◦), or more precisely: (1 + j), (−1 + j), (−1 − j), and (1 − j). These points correspond to combinations of the in-phase (I) and quadrature (Q) components, being either +1 or −1, and the respective signal constellations were already drawn in the design part of the document.

- In demodulation, the multiplication with sine/cosine followed by averaging over a carrier time interval gives us two values: the real and the imaginary part; these values decide what the message symbol corresponds to.
- The comparison uses the minimum Euclidean Distance criterion, through which the receiver can detect the original message.
- The bit and symbol error rates were calculated and plotted individually with SNR. By dividing the bit error rate by the number of bits in a symbol, we get the symbol error rate.
- As observed in the error rate Vs. SNR plot shows that with increasing SNR, there is a decrease in the error rates, which is non-linear throughout its range.
- They reach a point in SNR where below, it is completely not feasible to use the decoder as, due to low SNR, there is a high error rate.